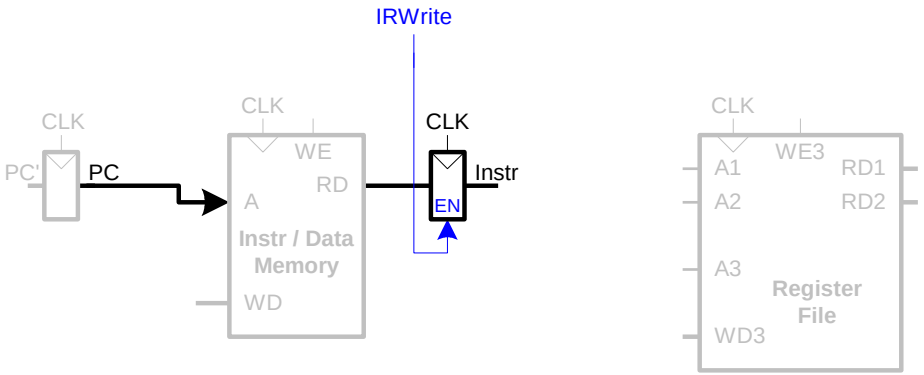
Let's Construct the Multi-Cycle Datapath

Consider the lw Instruction

- **For an instruction such as: `lw $t0, 0x20($t1)`**
- **We need to:**
  - Read the instruction from memory
  - Then read **`$t1`** from register array
  - Add the immediate value (**`0x20`**) to calculate the memory address
  - Read the content of this address
  - Write to the register **`$t0`** this content

Multi-Cycle Datapath: Instruction Fetch

- **We will consider lw, but fetch is the same for all instructions**
  - STEP 1: Fetch instruction

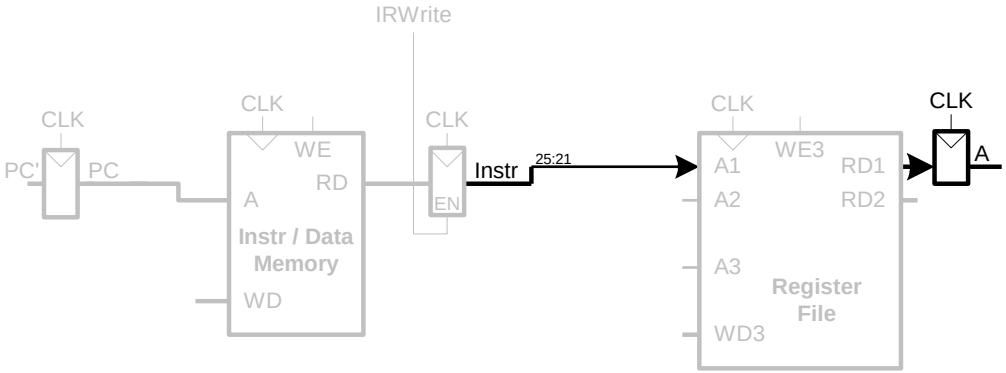


`read from the memory location [rs]+imm to location [rt]`

I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

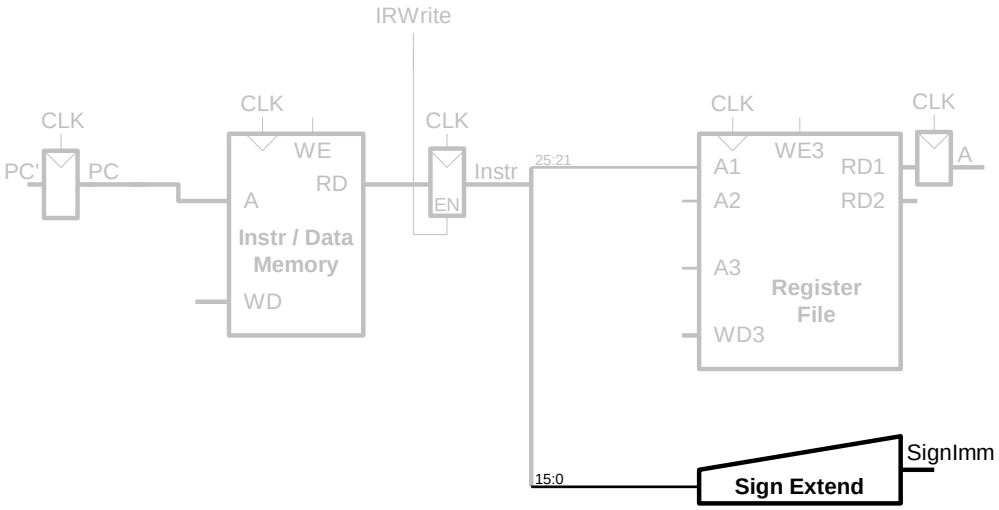
Multi-Cycle Datapath: lw register read



I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

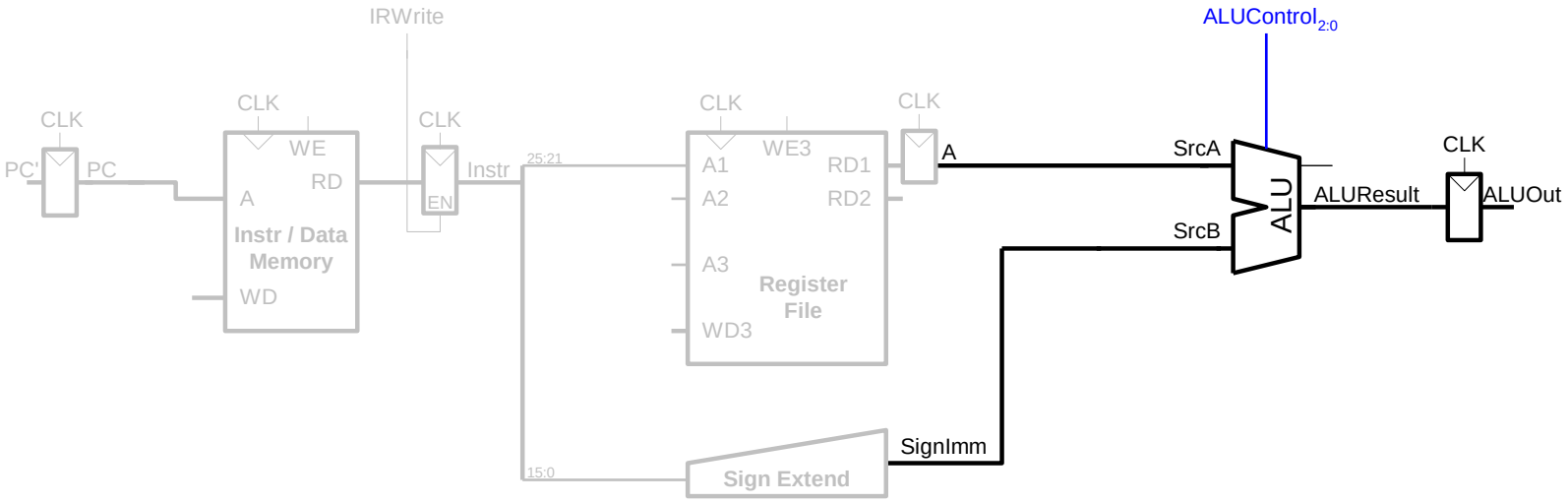
Multi-Cycle Datapath: lw immediate



I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

Multi-Cycle Datapath: lw address

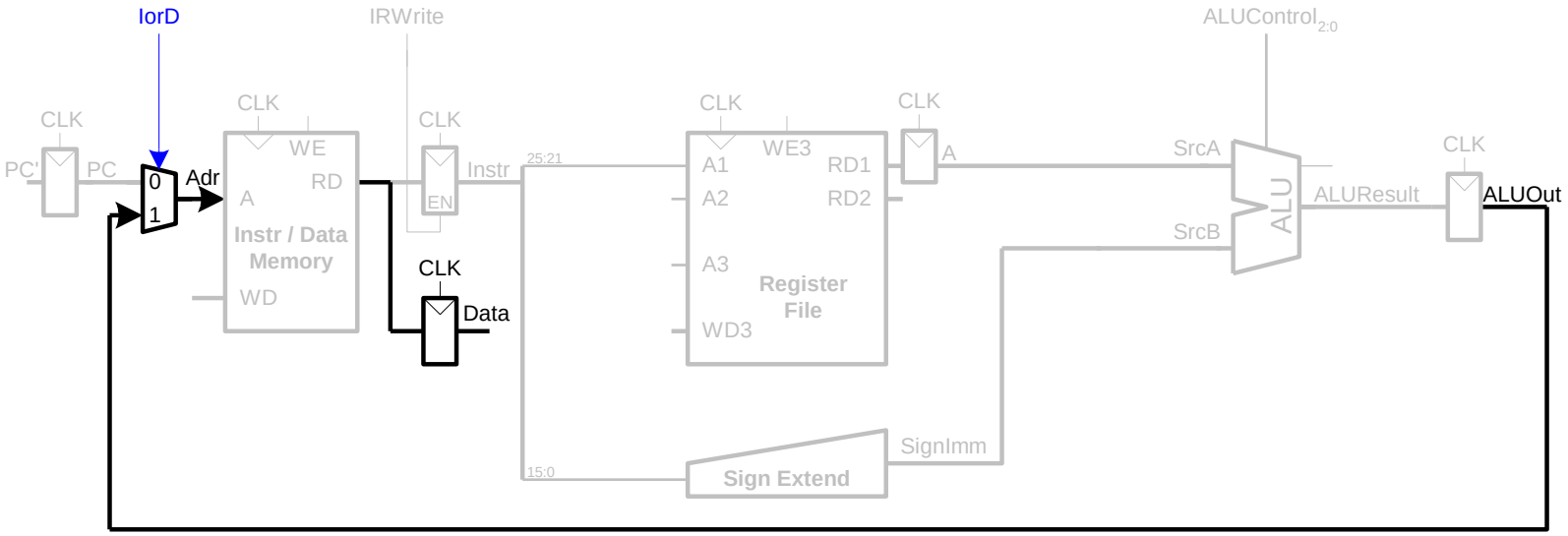


I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

Multi-Cycle Datapath: lw memory read

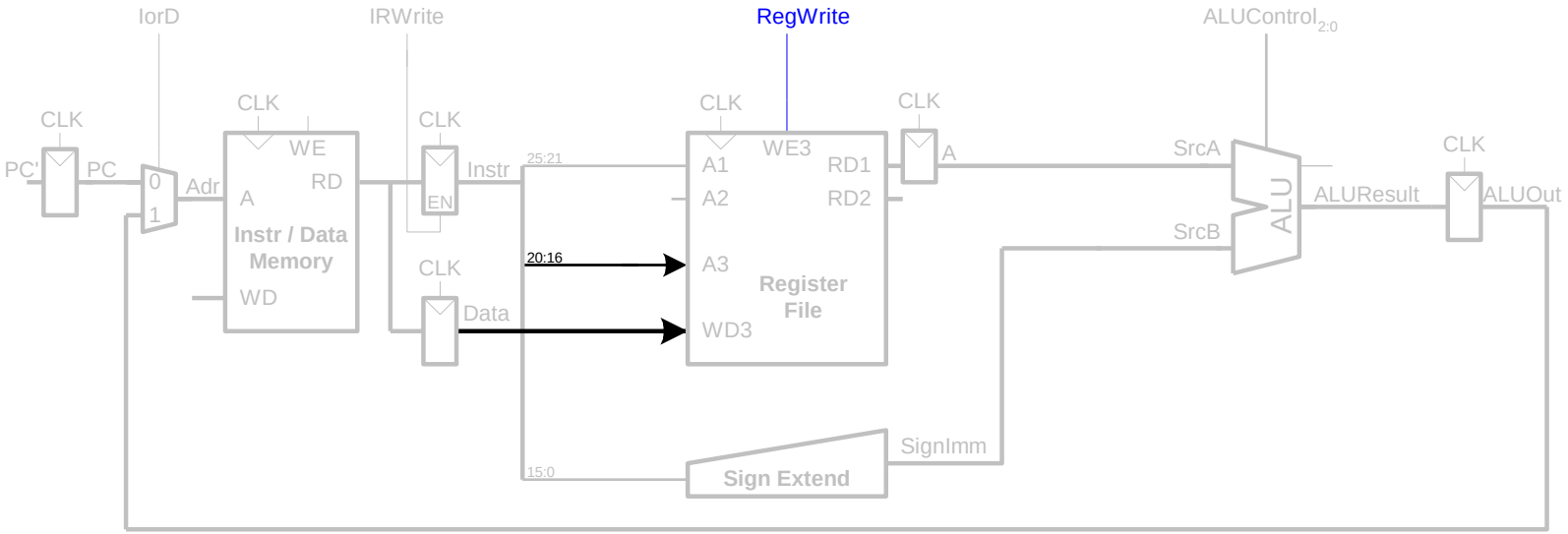
Now we can use **a single memory** to
both fetch an instruction and fetch an operand
(in different clock cycles)



I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

Multi-Cycle Datapath: lw write register

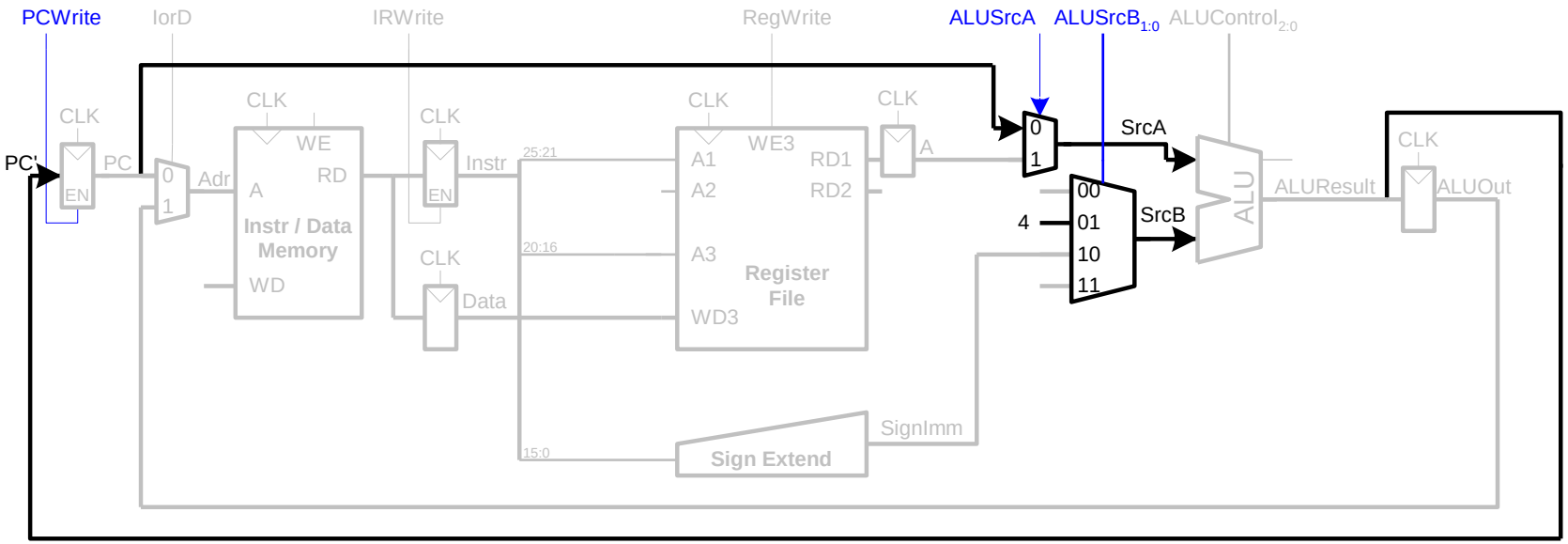


I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

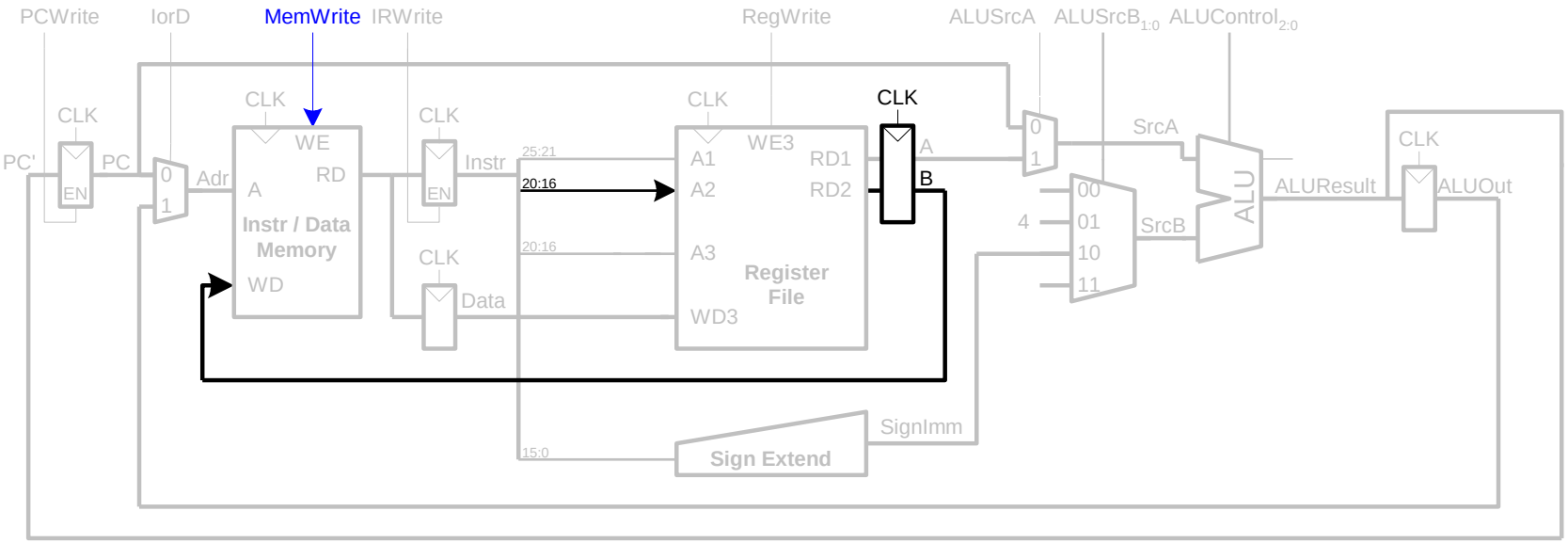
Multi-Cycle Datapath: increment PC

Now we can use **a single ALU** to both increment PC and do address calculation or arithmetic operations (in different clock cycles)



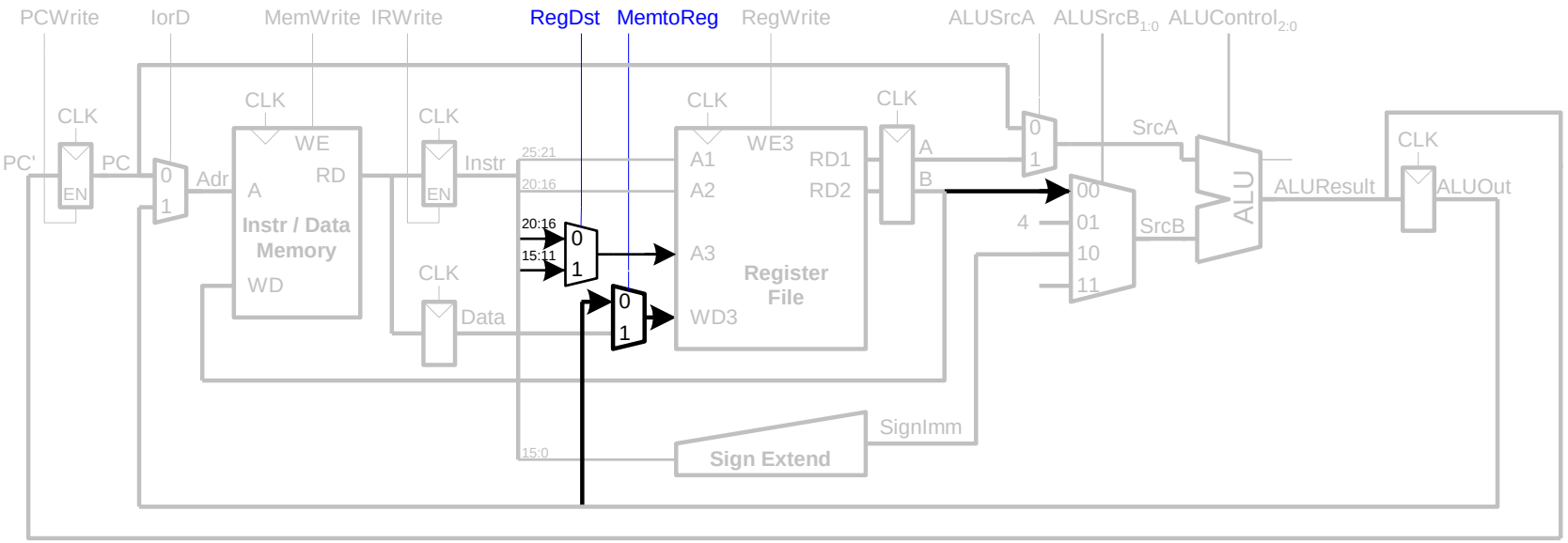
Multi-Cycle Datapath: sw

- **Write data in rt to memory**



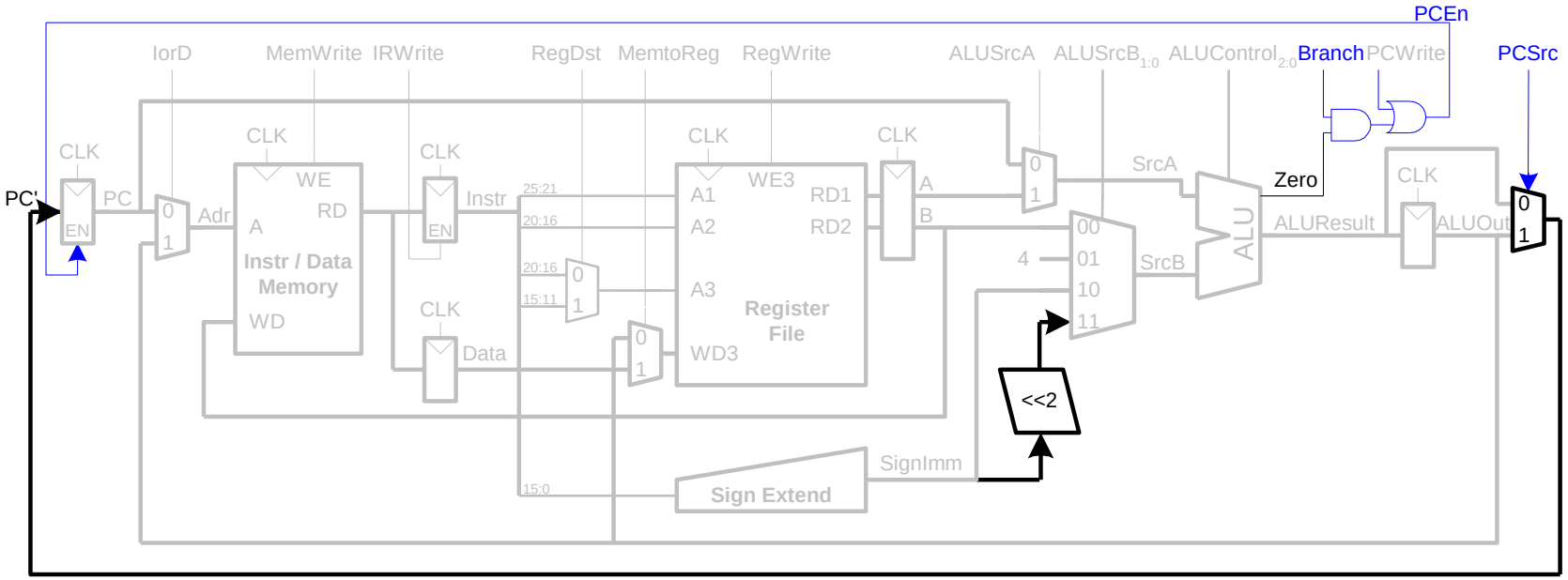
Multi-Cycle Datapath: R-type Instructions

- **Read from rs and rt**
  - Write ALUOut to register file
  - Write to rd (instead of rt)

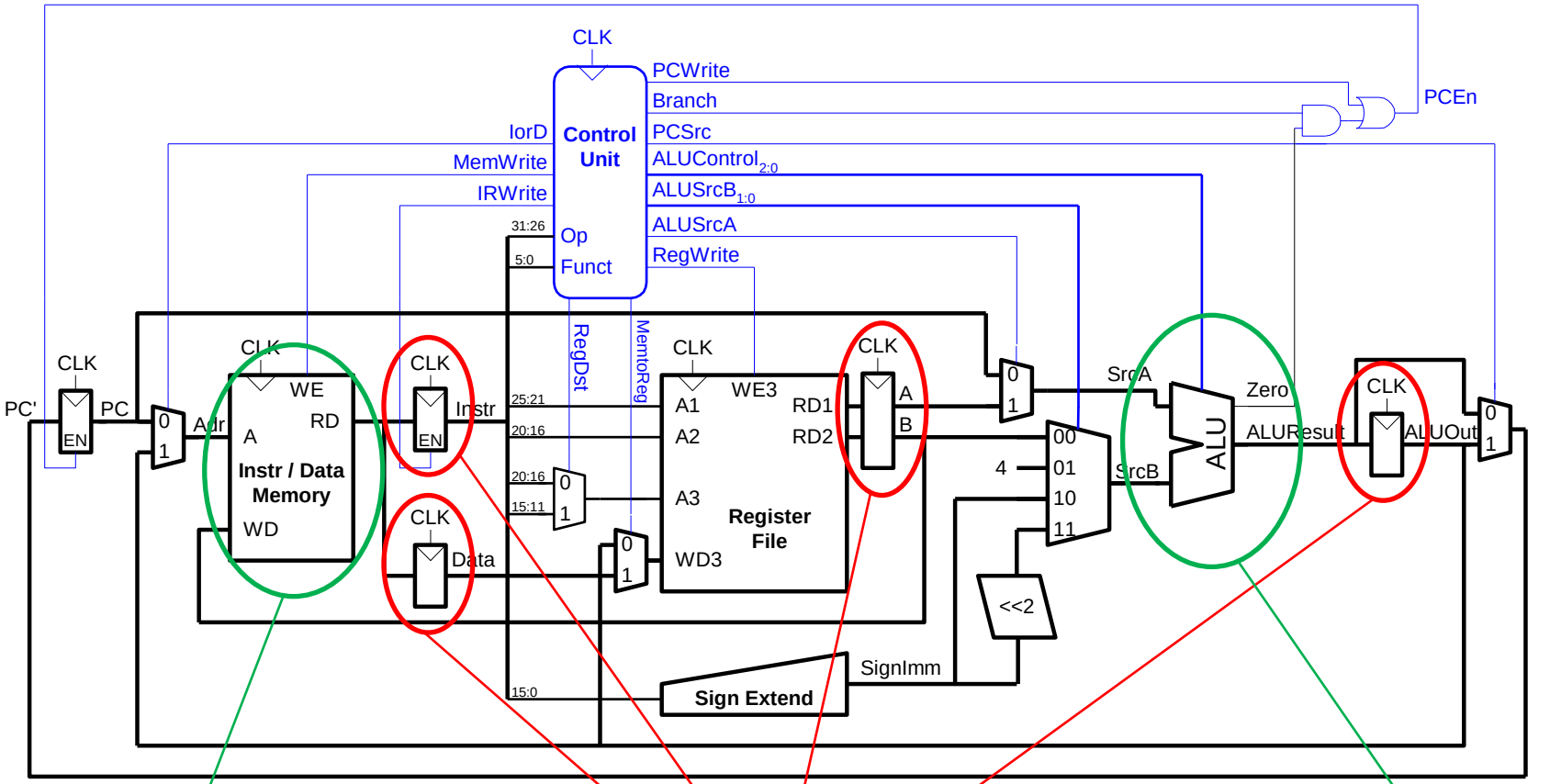


Multi-Cycle Datapath: beq

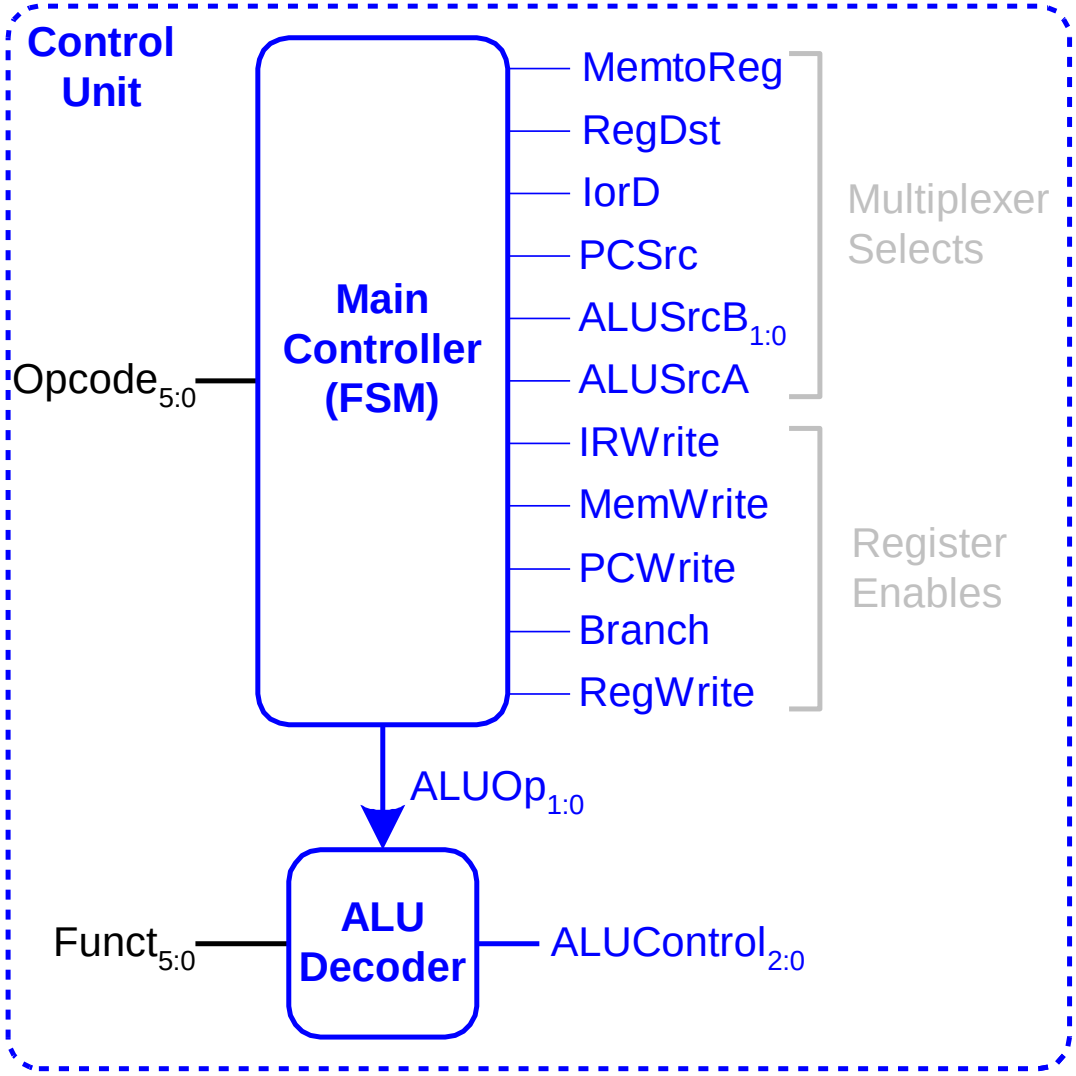
- **Determine whether values in rs and rt are equal**
  - Calculate branch target address:
    Target Address = (sign-extended immediate << 2) + (PC+4)



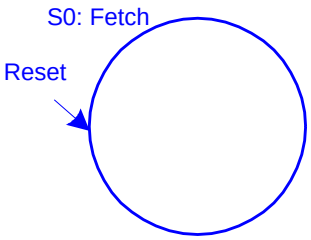
Complete Multi-Cycle Processor

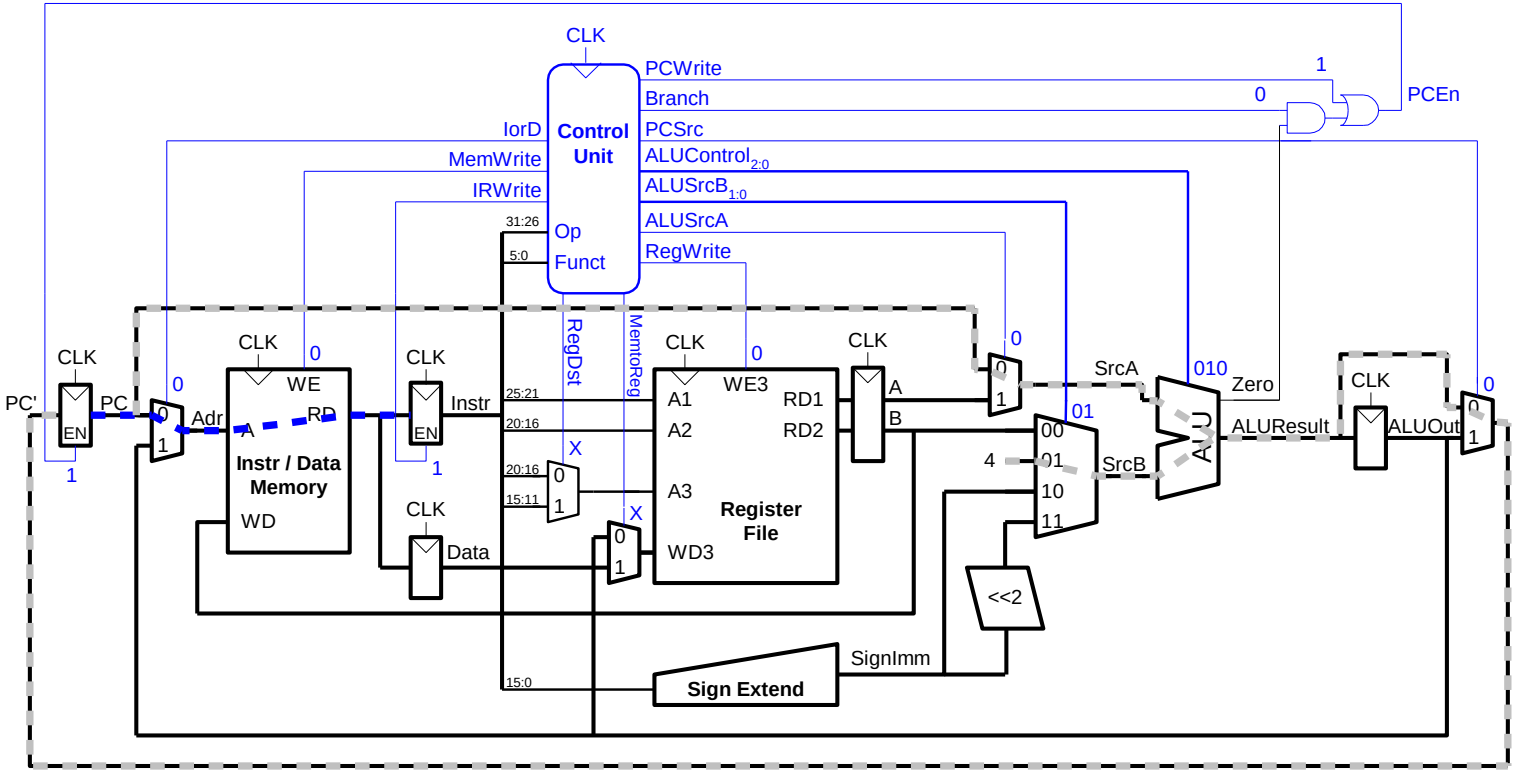


Control Unit

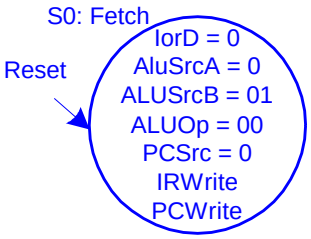


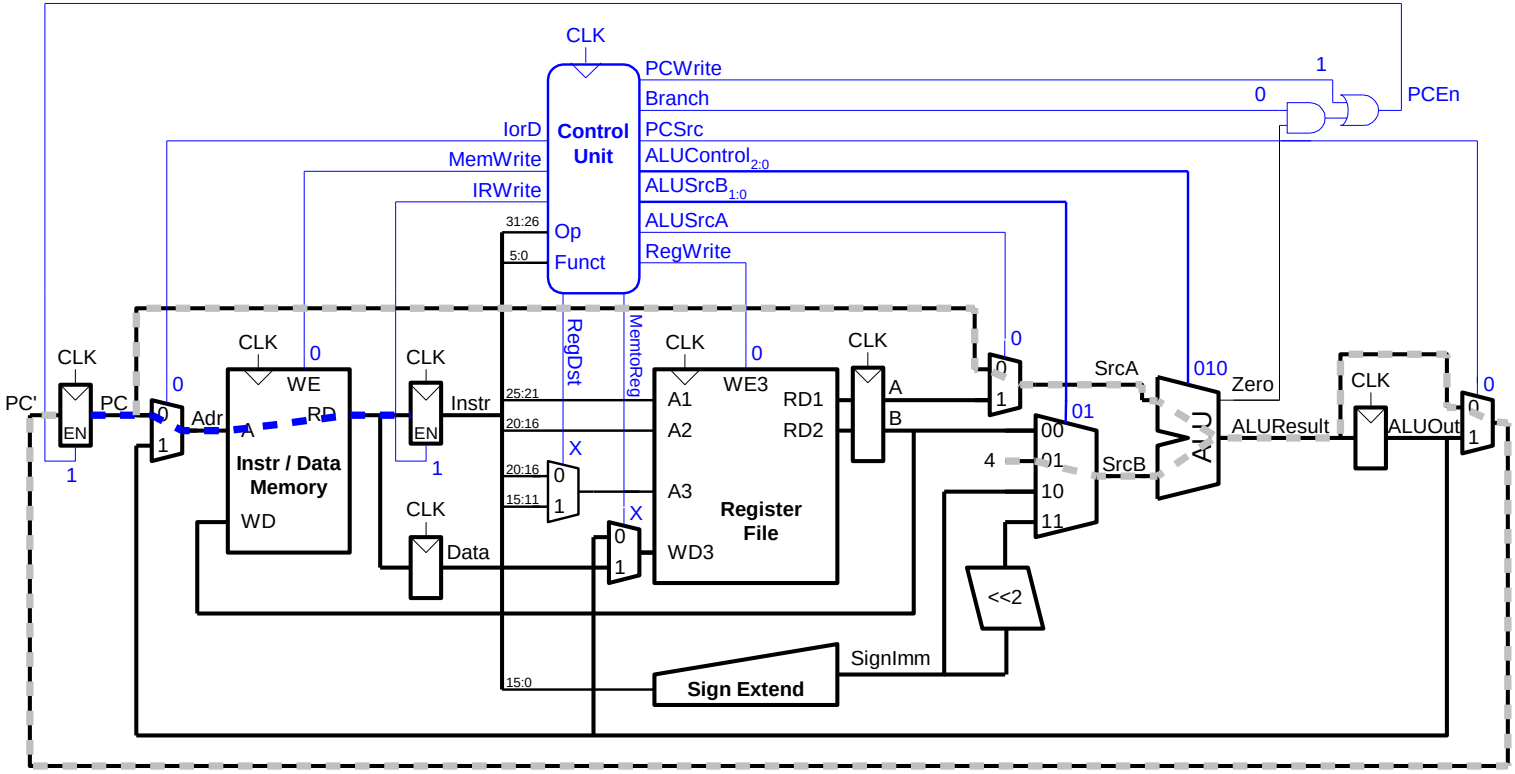
Main Controller FSM: Fetch



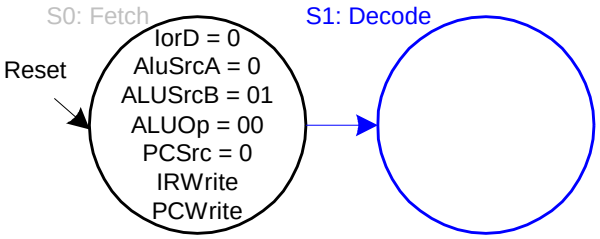


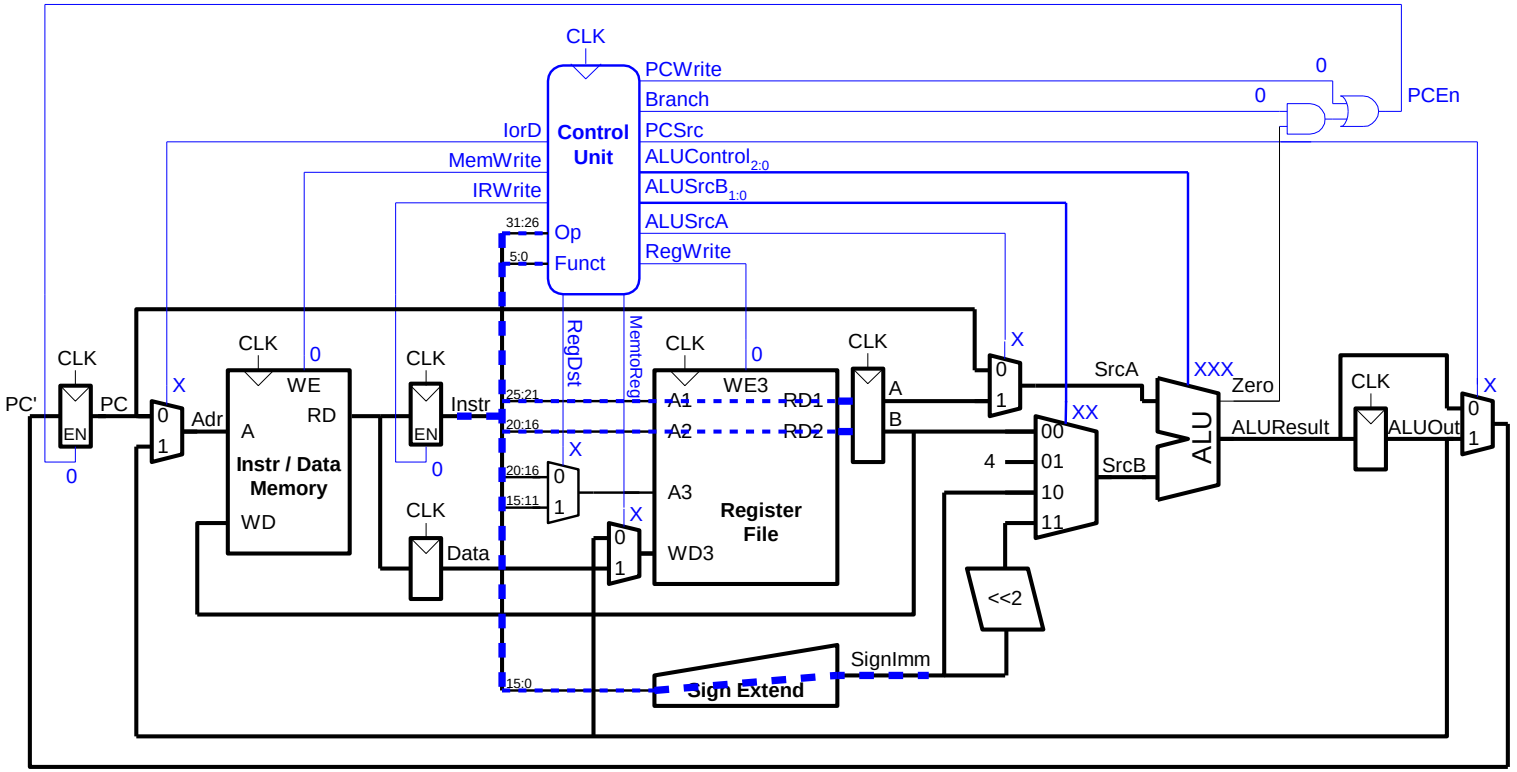
Main Controller FSM: Fetch



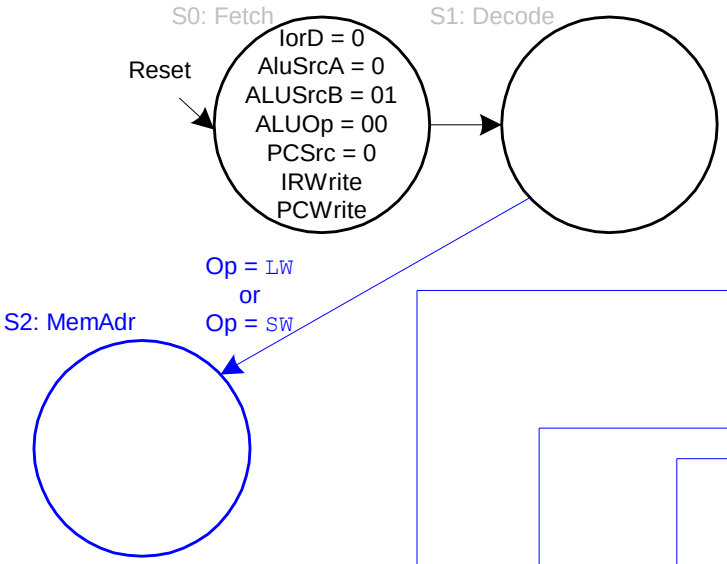


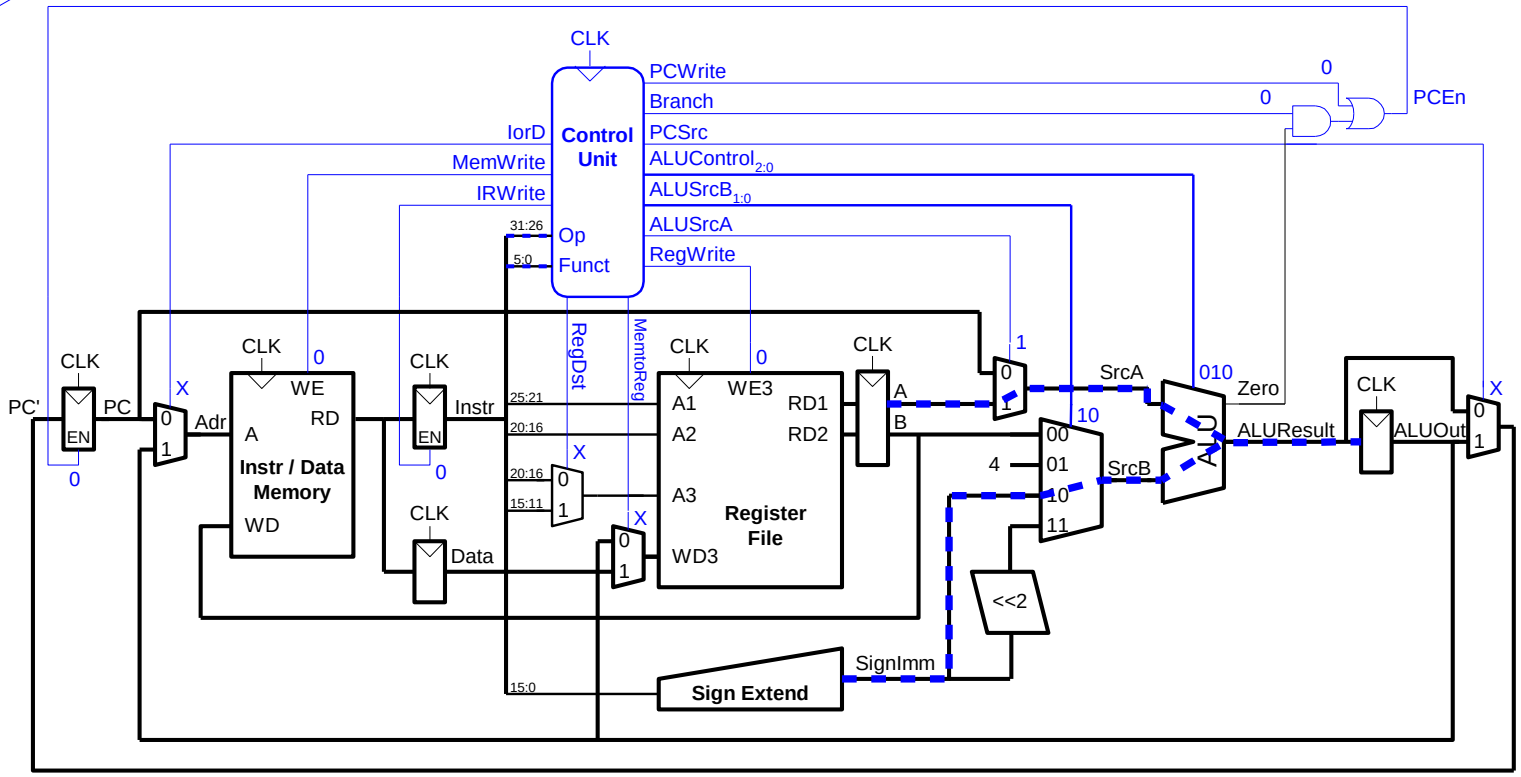
Main Controller FSM: Decode



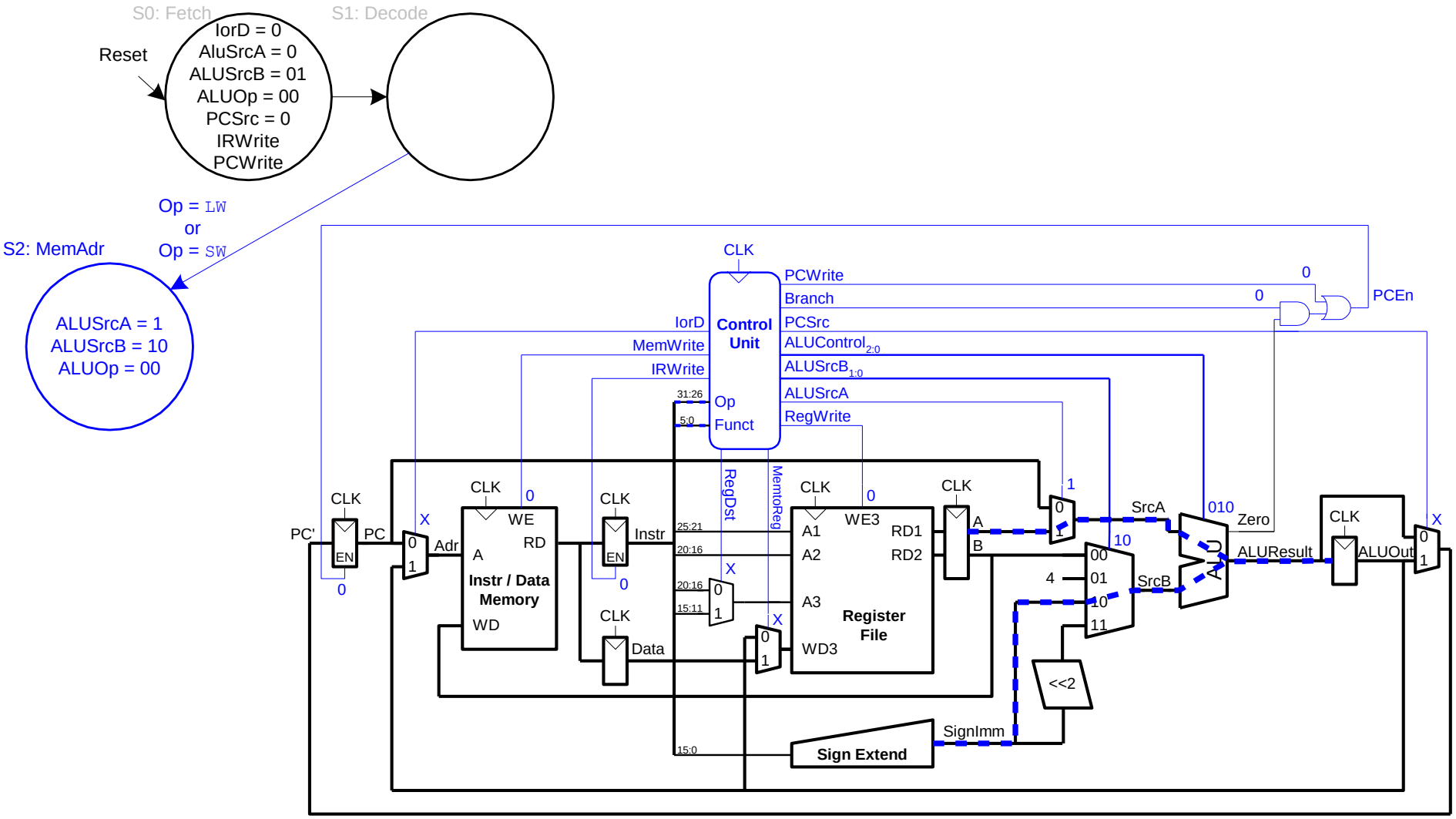


Main Controller FSM: Address Calculation

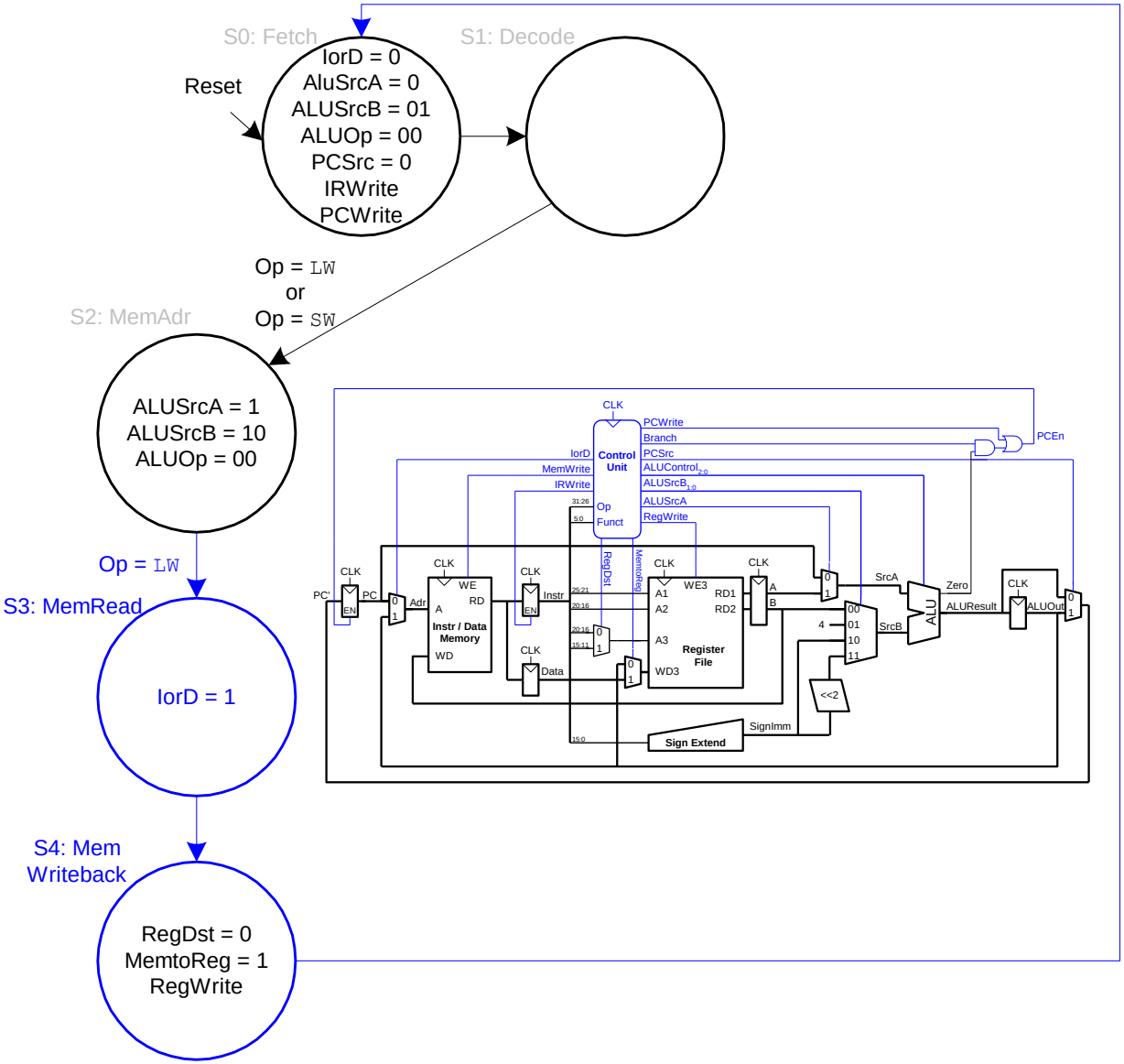




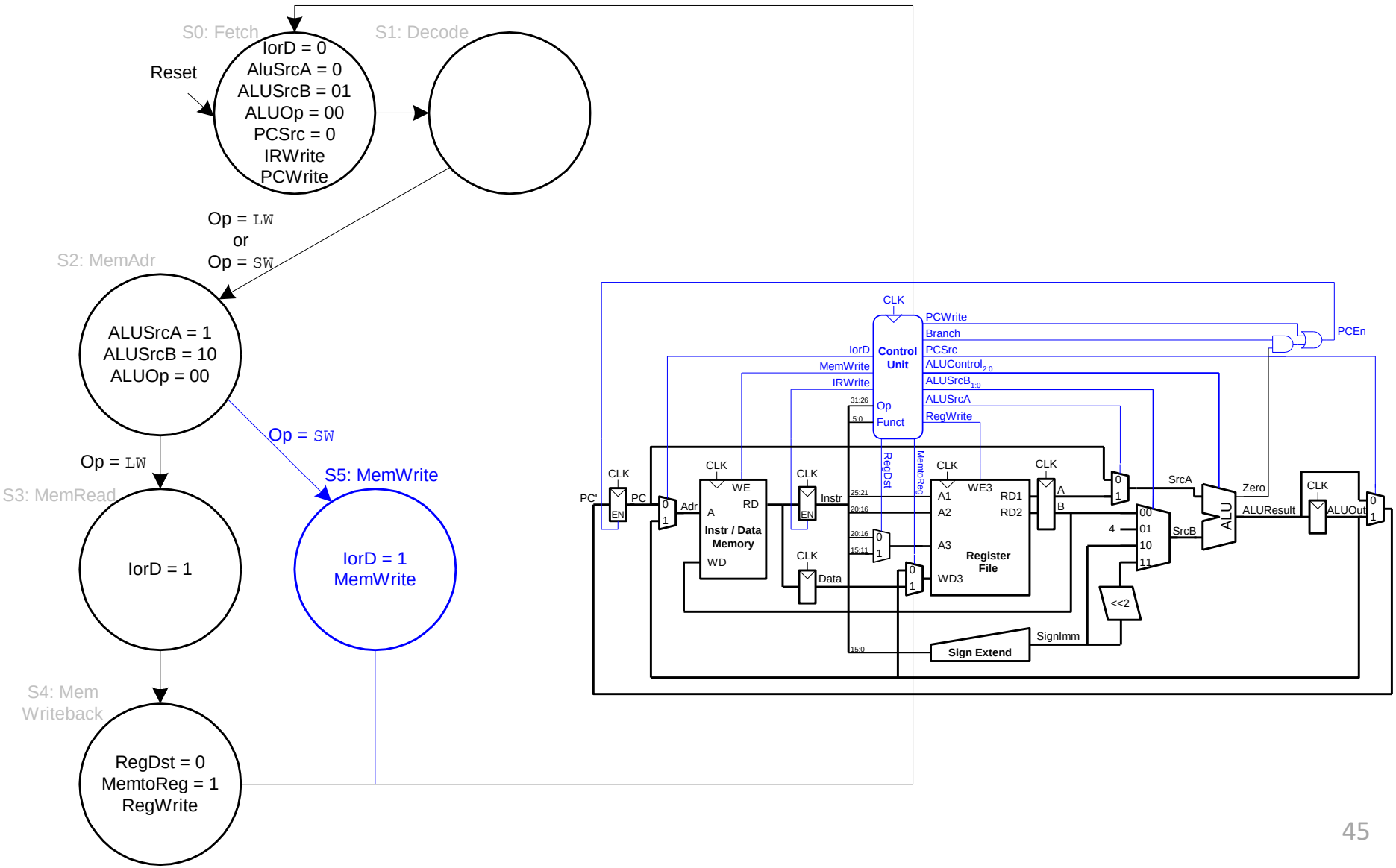
Main Controller FSM: Address Calculation



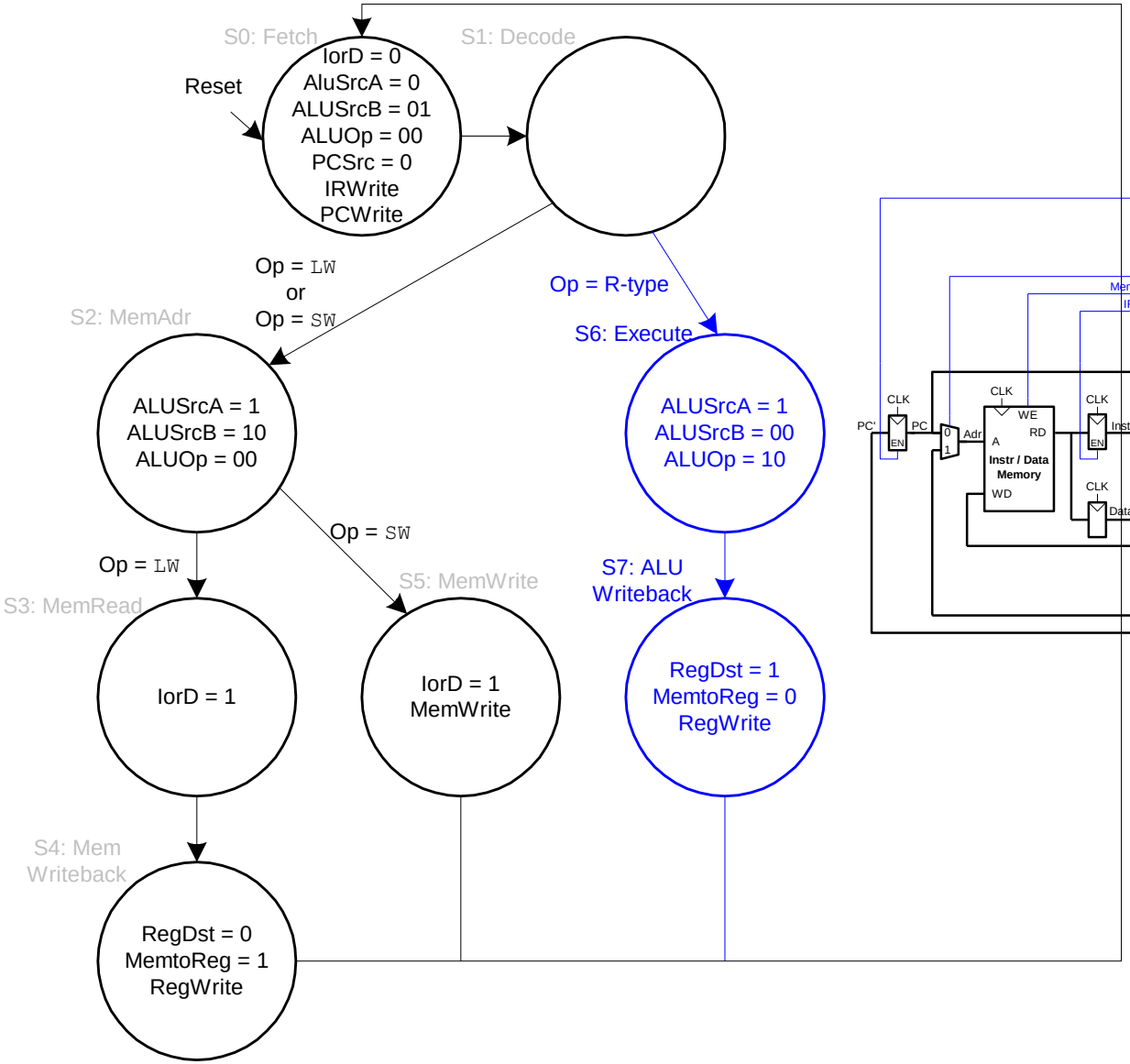
Main Controller FSM: `lw`

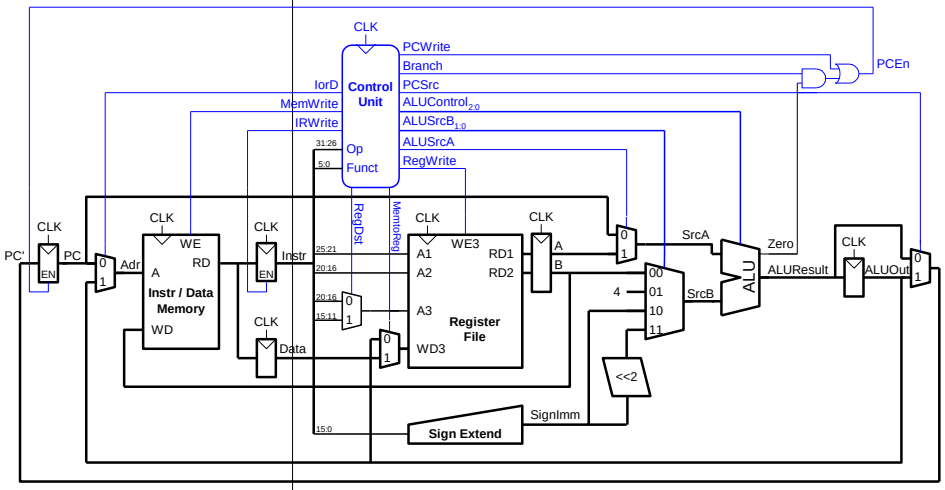


Main Controller FSM: sw

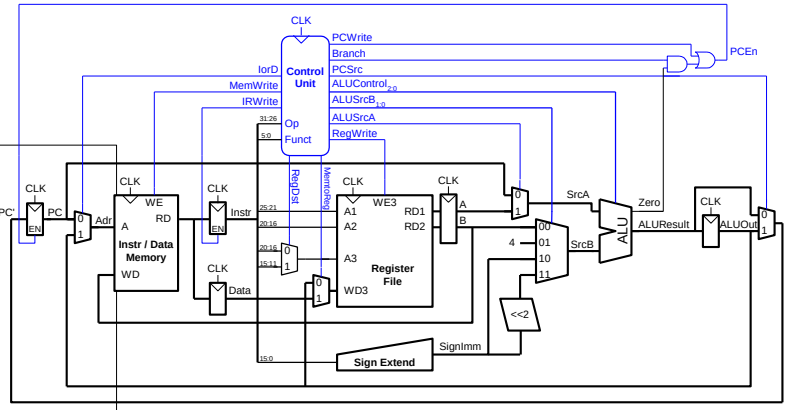


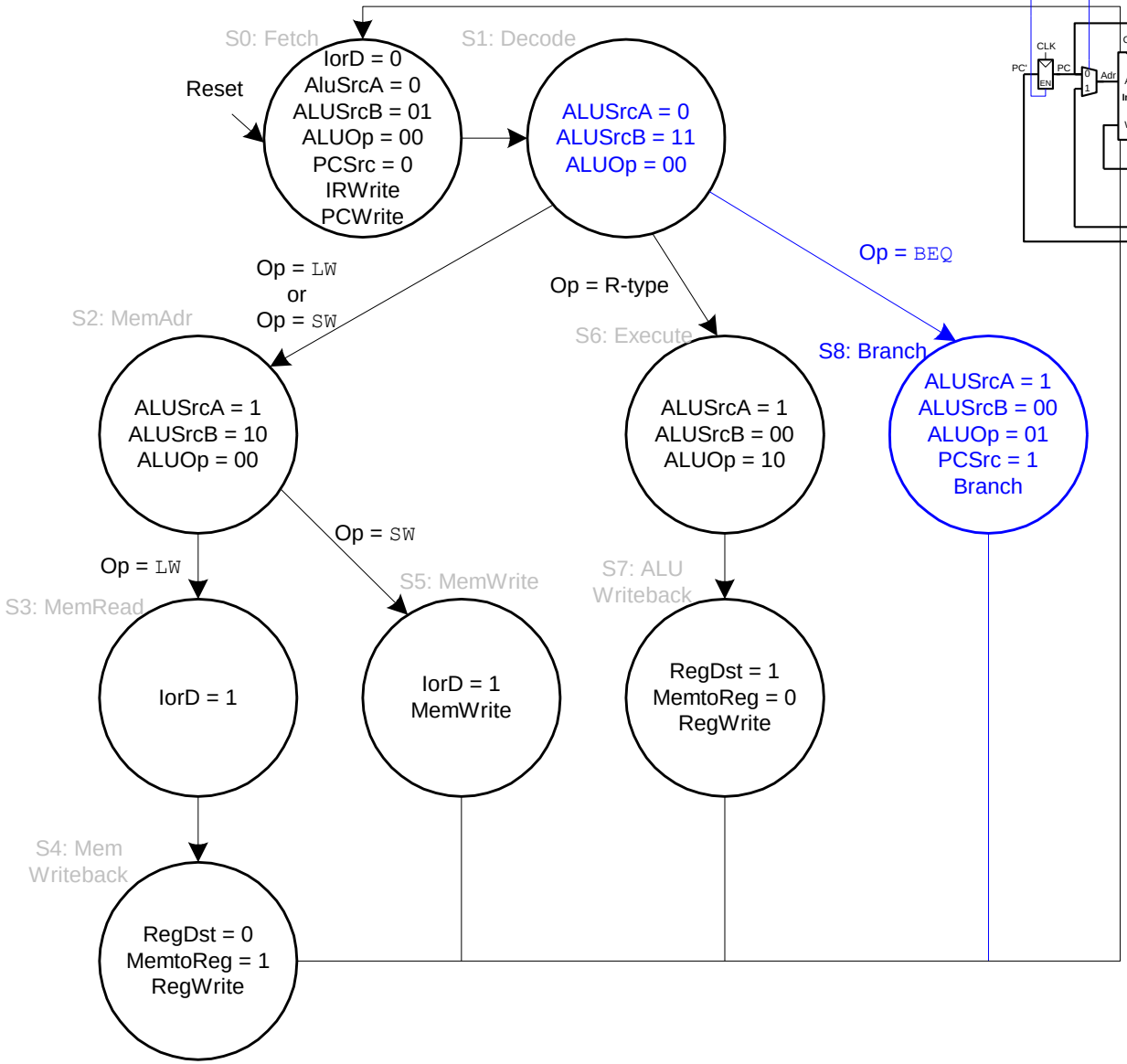
Main Controller FSM: R-Type



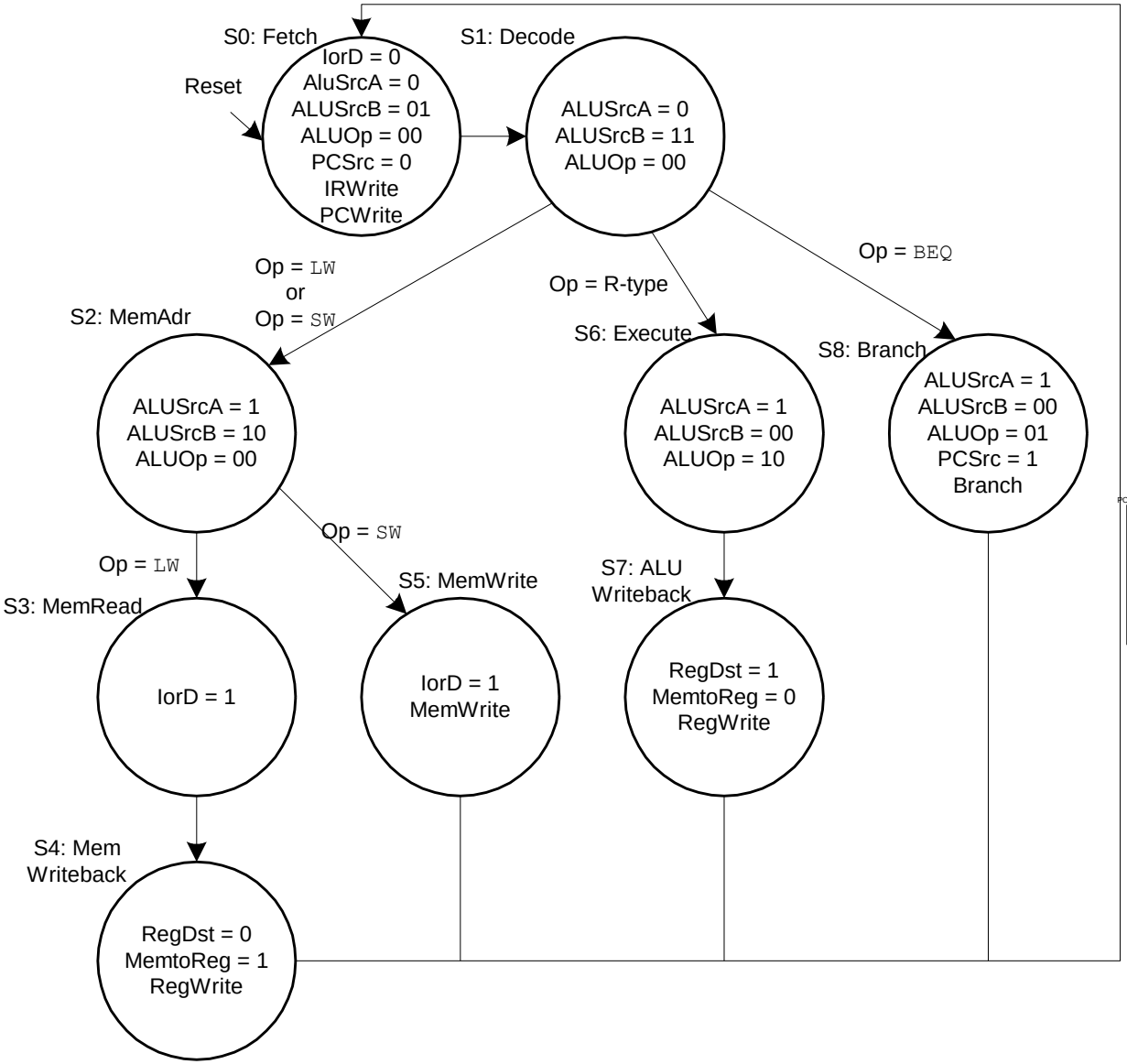


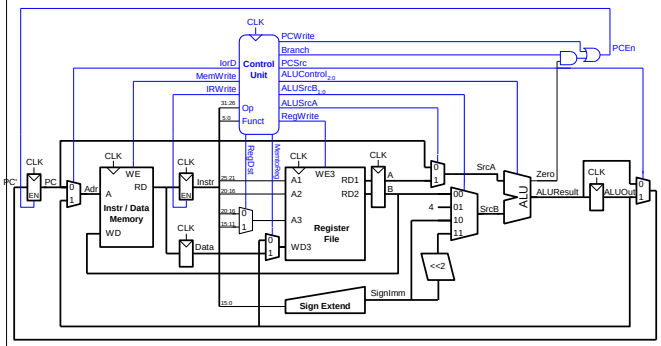
Main Controller FSM: beq



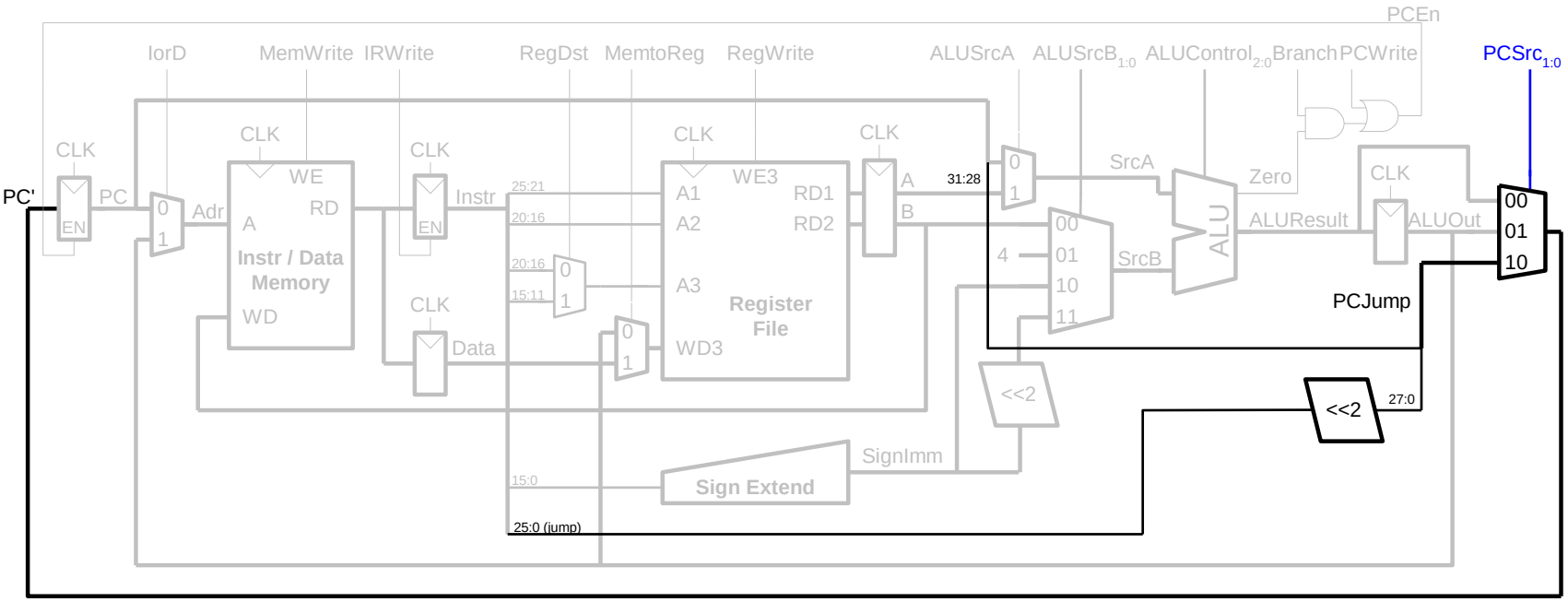


Complete Multi-Cycle Controller FSM

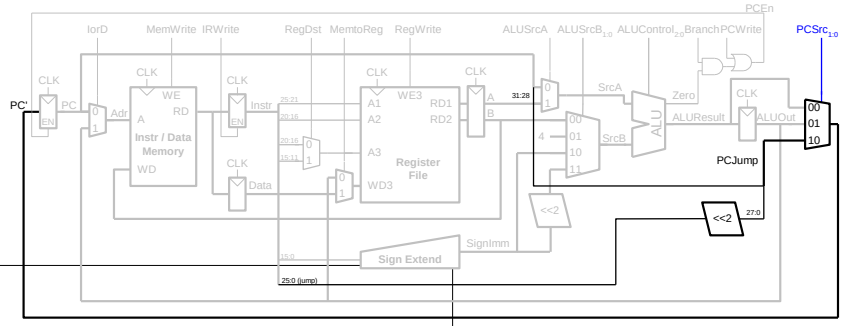


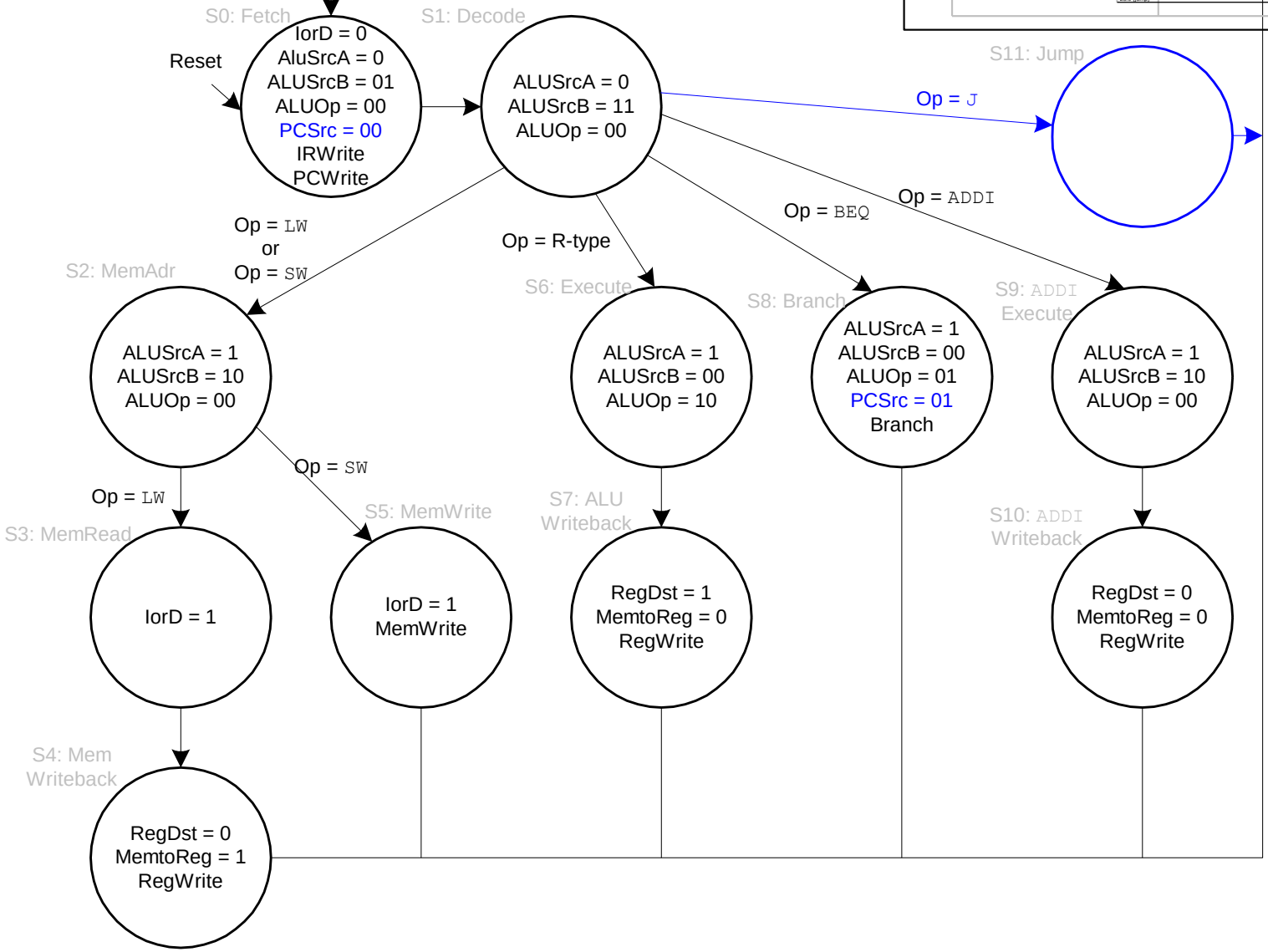


Extended Functionality: j

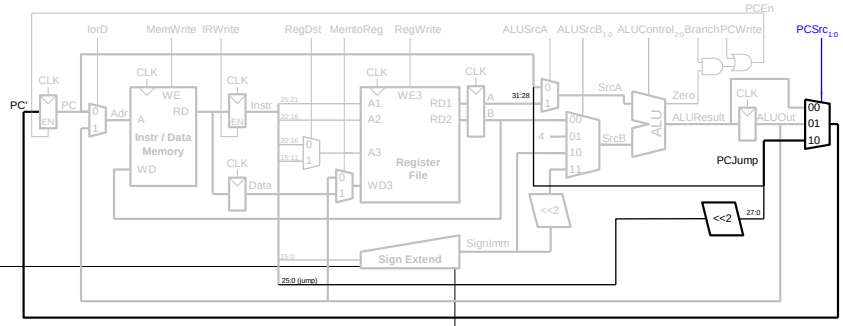


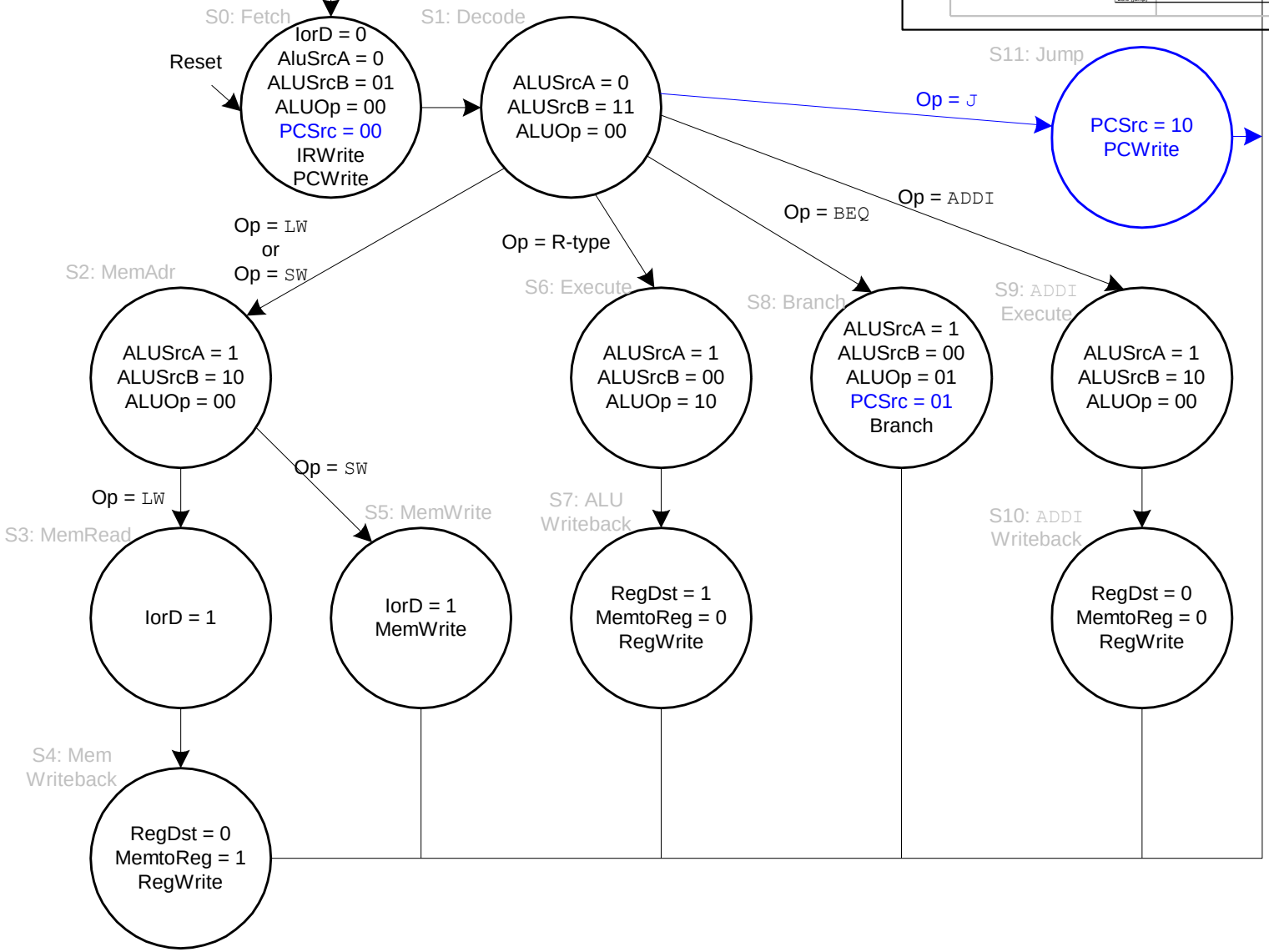
Control FSM: j





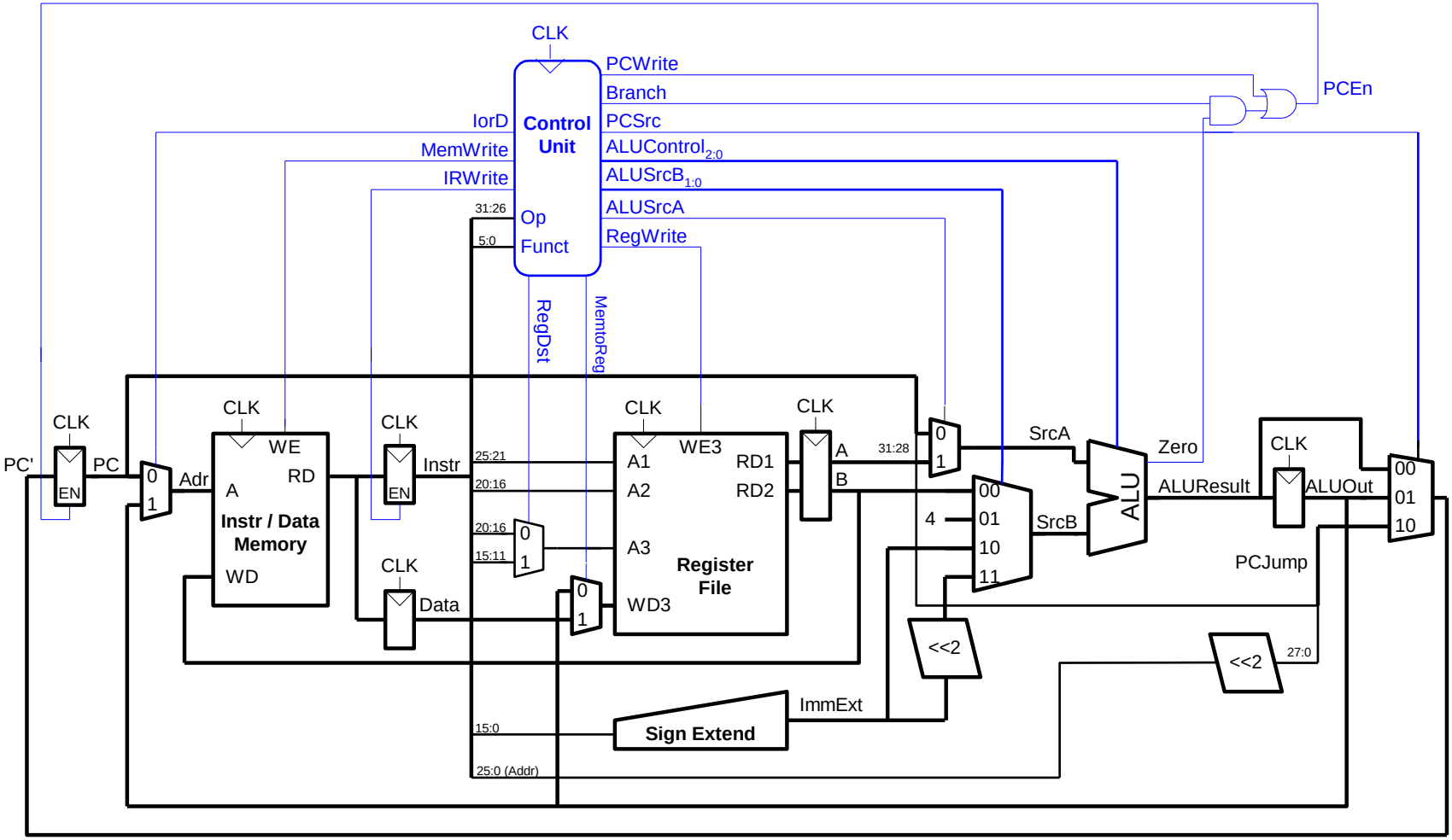
Control FSM: j





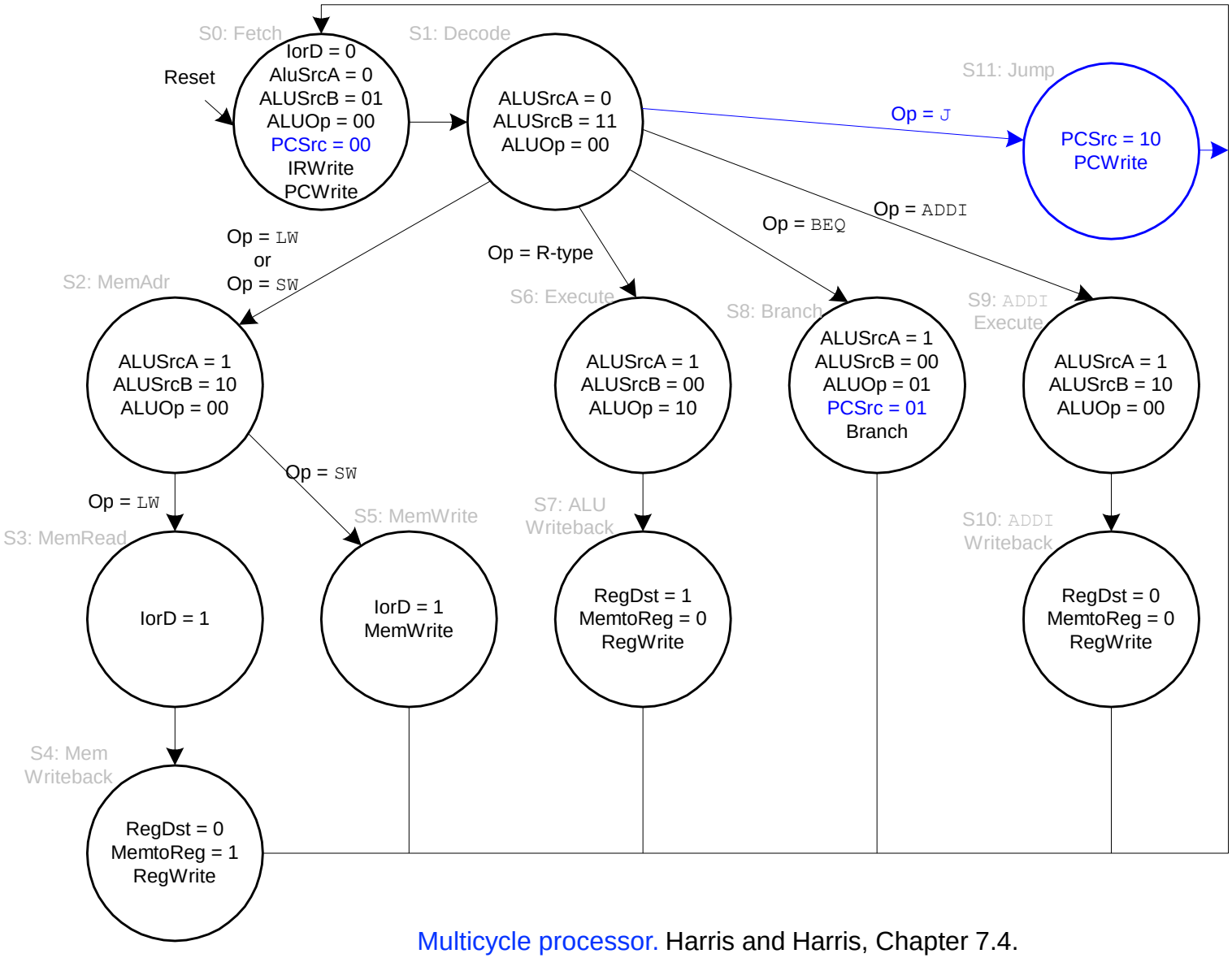
We Constructed a Multi-Cycle MIPS Microarchitecture

Review: Multi-Cycle MIPS Microarchitecture



Multicycle processor. Harris and Harris, Chapter 7.4.

Review: Multi-Cycle MIPS FSM



What does this design assume about memory?

Multicycle processor. Harris and Harris, Chapter 7.4.

Example Single-Cycle Datapath Analysis

- Assume (for the design in the previous slide)
  - memory units (read or write): 200 ps
  - ALU and adders: 100 ps
  - register file (read or write): 50 ps
  - other logic or wire delay: 0 ps

steps	IF	ID	EX	MEM	WB	Delay
resources	mem	RF	ALU	mem	RF	
R-type	200	50	100		50	400
I-type	200	50	100		50	400
LW	200	50	100	200	50	600
SW	200	50	100	200		550
Branch	200	50	100			350
Jump	200					200